

Disclaimer: Bagian ini ditulis dalam bahasa Indonesia yang santai sebagai **catatan pembelajaran**, bukan teks laporan formal. Tujuannya agar kamu benar-benar paham logikanya sebelum menulis laporan sendiri. Framework yang dipakai adalah **PyTorch + HuggingFace Transformers**, dan arsitekturnya adalah **LABAN (Label-Aware BERT Attention Network)** dengan backbone **IndoBERT** (`indobenchmark/indobert-base-p1`).

Apa yang Berubah dari LABAN Asli?

Sebelum masuk ke sub-bab, penting buat kamu pahami dulu: LABAN yang kamu pakai di proyek ini **bukan persis LABAN versi GitHub-nya**. Ada beberapa modifikasi yang dilakukan agar model ini bisa berjalan di chatbot konseling Alkitab berbahasa Indonesia. Berikut penjelasan perubahan-perubahannya.

Tentang LABAN Asli

LABAN singkatnya adalah **Label-Aware BERT Attention Network**, sebuah arsitektur yang dipublikasikan di EMNLP 2021 oleh Wu et al. Ide besarnya sederhana tapi cerdas: daripada pakai *linear head* biasa (yang harus di-retrain kalau ada intent baru), LABAN menggunakan **dua encoder BERT secara bersamaan** — satu untuk membaca kalimat pengguna, satu lagi untuk membaca teks label intent. Kedua outputnya kemudian dibandingkan secara matematis. Karena label dibaca sebagai teks, model ini bisa secara teori mengenali intent yang belum pernah dilihat sebelumnya (*zero-shot*).

Modifikasi 1: Ganti Backbone dari English BERT ke IndoBERT

LABAN aslinya *hardcoded* pakai `bert-base-uncased`, yaitu BERT versi bahasa Inggris. Masalahnya, semua data kita — input pengguna, label intent, dataset training — semuanya **Bahasa Indonesia**. Kalau dipaksakan pakai BERT Inggris, modelnya akan kesulitan memahami semantik bahasa kita.

Solusinya: backbone diganti ke **IndoBERT** (`indobenchmark/indobert-base-p1`) yang memang dilatih pada corpus Bahasa Indonesia. Supaya lebih fleksibel (karena kita juga perlu membandingkan beberapa backbone dalam evaluasi), kodenya menggunakan `AutoModel` dan `AutoTokenizer` dari HuggingFace — dengan ini, kita tinggal ganti satu baris di `config.py` untuk mencoba backbone lain tanpa ubah kode model sama sekali.

Modifikasi 2: Arsitektur Dipangkas Habis

LABAN asli di GitHub itu kompleks banget — ada **6 mode surface encoder** dan **6 mode label-aware layer**, total ada puluhan komponen yang bisa dikonfigurasi. Ini wajar karena LABAN itu riset akademik yang membandingkan banyak varian. Tapi untuk chatbot produksi, kita tidak butuh semuanya.

Proyek ini hanya mengambil **satu mode yang relevan**: `surface encoder normal` (pakai `pooler_output` token `[CLS]`) dan `label-aware layer zero-shot` (proyeksi invers matriks

Gram). Hasilnya, kelas model yang tadinya ~227 baris **dipotong jadi ~45 baris**. Layer-layer yang dibuang antara lain: `self.dropout`, `self.classifier`, `self.mapping`, `self.relations1`, `self.relations2`, dan semua switch `self.mode/self.mode2`.

Modifikasi 3: Regularisasi Tikhonov untuk Stabilitas Numerik

LABAN asli melakukan inversi matriks Gram secara langsung dengan `torch.inverse()` tanpa pengaman apapun. Ini berbahaya: kalau dua label intent semantiknya terlalu mirip (misalnya "Sedih dan Kehilangan" dengan "Takut dan Kecemasan"), matriks Gram bisa menjadi *singular* dan `torch.inverse()` akan menghasilkan nilai NaN atau bahkan crash.

Proyek ini menambahkan **regularisasi Tikhonov** — yaitu menambahkan matriks identitas kecil ($\epsilon * I$ di mana $\epsilon = 1e-4$) sebelum inversi dilakukan. Ini menjamin matriks Gram selalu *invertible*.

```
# LABAN Asli (berbahaya):
logits = torch.mm(weight, torch.inverse(gram)) * np.sqrt(768)

# Proyek ini (dengan regularisasi Tikhonov):
gram = gram + torch.eye(gram.size(0), device=gram.device) * 1e-4
logits = torch.mm(weight, torch.inverse(gram)) * np.sqrt(opt.hidden_size)
```

Modifikasi 4: Format Data Disesuaikan

LABAN asli bergantung pada file `.pkl` (pickle) dengan struktur khusus untuk dataset benchmark Inggris (MixATIS, MixSNIPS, dll.). Proyek ini membangun pipeline data sendiri dari nol menggunakan **CSV dengan format multi-label** berbasis tanda titik koma (;), sesuai dataset konseling Alkitab kita.

Modifikasi 5: Pipeline Training yang Lebih Ketat (Thesis-Grade)

LABAN asli tidak punya validasi per-epoch atau *best-checkpoint saving*. Proyek ini menambahkan split 80/10/10 (train/validasi/test), validasi F1 setiap epoch, menyimpan checkpoint terbaik berdasarkan F1 validasi tertinggi, dan evaluasi akhir pada test set yang benar-benar terpisah.

Aspek	LABAN Asli (GitHub)	Proyek Ini
Bahasa	Inggris (bert-base-uncased)	Indonesia (indobenchmark/indobert-base-p1)
Backbone	Hardcoded	Configurable via <code>AutoModel + config.py</code>
Arsitektur	6 mode surface, 6 mode label	Hanya <code>normal + zero-shot</code>
Layer Tambahan	Dropout, classifier, mapping, relations	Tidak ada (murni zero-shot projection)
Stabilitas Numerik	Tidak ada regularisasi	Tikhonov regularization ($\epsilon = 1e-4$)
Format Data	File Pickle (<code>.pkl</code>)	CSV dengan multi-label dipisah titik koma
Data Split	Tidak ada validasi formal	80/10/10 train/val/test

Aspek	LABAN Asli (GitHub)	Proyek Ini
Checkpointing	Simpan model akhir	Simpan model dengan F1 validasi terbaik
Ukuran Kode Model	~227 baris	~45 baris

Penjelasan Per Sub-Bab (Bahasa Indonesia Santai + Kode PyTorch)

3.5.1 Input Kueri Pengguna

WHAT — Apa yang terjadi?

Ini adalah titik masuk pertama. Pengguna mengetik sesuatu di kotak chat — misalnya *"Aku merasa sangat sedih dan tidak tahu harus berbuat apa"* — dan sistem menerimanya sebagai teks mentah (raw string). Di tahap ini, belum ada pemrosesan berat. Tugas utamanya hanya memastikan teks itu "bersih" dan siap diteruskan ke mesin tokenizer.

Secara teknis, sistem menerima Python string dari Flask (`request.json['message']`), lalu teks ini diteruskan langsung ke pipeline LABAN. Tidak ada stemming, tidak ada stopword removal, tidak ada pembersihan tanda baca — karena BERT-based model justru butuh teks sealami mungkin untuk bisa menangkap nuansa emosional yang ada di dalamnya.

WHY — Kenapa penting?

Kalimat pengguna di chatbot konseling biasanya mengandung emosi yang kaya, slang, atau bahkan kalimat yang tidak lengkap. Kalau kita agresif melakukan preprocessing (misalnya buang semua tanda baca atau kata-kata pendek), kita bisa kehilangan sinyal emosional yang justru menjadi inti dari klasifikasi. Model BERT sudah dirancang untuk menerima teks mentah — jadi biarkanlah BERT yang "mencerna" kompleksitas itu.

HOW — Bagaimana datanya berubah?

Input: "Aku merasa sangat sedih dan tidak tahu harus berbuat apa" Output: String Python yang siap diteruskan ke tokenizer Shape: Tidak ada perubahan shape di tahap ini — masih string biasa

```
# Di app.py / session_manager.py – menerima input dari Flask
user_message = request.json.get('message', '').strip()

# Teks ini langsung diteruskan ke predictor tanpa modifikasi besar
# (Pembersihan minimal: cukup .strip() saja)
result = predictor.predict(user_message)
```

3.5.2 EnkodeInput

WHAT — Apa yang terjadi?

Sekarang teks mentah itu diubah jadi angka. Proses ini disebut **tokenisasi**. Kita menggunakan

`AutoTokenizer` dari HuggingFace yang sudah dilatih bersama model IndoBERT. Tokenizer ini menggunakan metode **WordPiece** — artinya kalau ada kata yang tidak dikenal (misalnya nama orang atau kata asing), kata itu akan dipecah jadi sub-kata yang lebih kecil supaya tidak terjadi *out-of-vocabulary*.

Panjang input dibatasi maksimal **50 token** (diset di `config.py` sebagai `max_len`). Kenapa 50? Karena kueri chatbot biasanya pendek, dan membatasi panjang ini menghemat memori GPU serta mempercepat inference.

Output tokenisasi ada dua: `input_ids` (urutan angka ID setiap token) dan `attention_mask` (angka 1 untuk token asli, 0 untuk padding).

WHY — Kenapa penting?

BERT tidak bisa membaca kata. BERT hanya bisa membaca angka dalam bentuk vektor. Tokenisasi adalah "kamus" yang memetakan kata ke angka berdasarkan vocabulary model. `attention_mask` penting agar model tahu mana token yang sebenarnya konten dan mana yang cuma padding pengisi.

HOW — Bagaimana datanya berubah?

Input: "Aku merasa sangat sedih" <- String biasa (Tokenisasi) `input_ids`: [2, 1045, 7903, 4999, 8765, 3, 0, 0, ..., 0] Shape: (1, 50) `attention_mask`: [1, 1, 1, 1, 1, 1, 0, 0, ..., 0] Shape: (1, 50)

```
from transformers import AutoTokenizer
from config import opt

tokenizer = AutoTokenizer.from_pretrained(opt.MODEL_NAME)

# Tokenisasi kueri pengguna
encoded = tokenizer(
    user_text,
    max_length=opt.max_len,          # Maksimal 50 token
    padding='max_length',            # Isi sisanya dengan [PAD]
    truncation=True,                 # Potong kalau lebih panjang
    return_tensors='pt'              # Kembalikan sebagai PyTorch tensor
)

utterance_ids = encoded['input_ids'] # Shape: (1, 50)
utterance_mask = encoded['attention_mask'] # Shape: (1, 50)
```

3.5.3 Self-AttentiveInput

WHAT — Apa yang terjadi?

Token-token yang sudah jadi angka tadi sekarang dimasukkan ke dalam **BERT encoder pertama** (`self.bert` di `bert_model.py`). Di sinilah keajaiban transformer bekerja. Model IndoBERT memiliki 12 lapisan *self-attention*, dan di setiap lapisannya, setiap token "berkomunikasi" dengan semua token lain dalam kalimat untuk memahami konteksnya.

Misalnya, kata "sedih" di kalimat "Aku merasa sedih karena kehilangan" akan mendapat perhatian lebih ke kata "kehilangan" (sebagai penyebab), bukan ke kata "karena" (hanya kata penghubung). Inilah *self-attention* — ia secara dinamis menentukan seberapa penting setiap kata terhadap kata lain.

Dari seluruh proses ini, kita hanya mengambil **pooler_output** — yaitu representasi dari token spesial [CLS] yang ada di posisi pertama setiap input BERT. Token [CLS] ini secara konvensi merepresentasikan "makna keseluruhan kalimat" dan merupakan output dengan dimensi 768.

WHY — Kenapa penting?

Untuk klasifikasi intent, kita butuh satu vektor yang merangkum seluruh makna kalimat, bukan vektor per-kata. **pooler_output** dari token [CLS] adalah representasi ringkasan itu.

Mekanisme self-attention memastikan representasi ringkasan ini kaya konteks — ia "tahu" bahwa kata "sedih" muncul dalam konteks kehilangan, bukan dalam konteks cerita lucu.

HOW — Bagaimana datanya berubah?

Input: utterance_ids Shape: (1, 50) utterance_mask Shape: (1, 50) (12 Lapisan Self-Attention BERT) pooler_output: Shape: (1, 768) <- Satu vektor berisi 768 angka yang merepresentasikan makna kalimat

```
# Di bert_model.py - BertEmbedding.forward()
output_utterance = self.bert(
    input_ids=utterance_ids,
    attention_mask=utterance_mask,
    output_hidden_states=True,
    output_attentions=True,
    return_dict=True
)

# Ambil representasi [CLS] token - ringkasan seluruh kalimat
pooled_output = output_utterance.pooler_output # Shape: (batch_size, 768)
```

3.5.4 InputLabel

WHAT — Apa yang terjadi?

Bersamaan dengan memproses kalimat pengguna, sistem juga mempersiapkan **10 teks label intent** yang kita punya. Label-label ini bukan angka — melainkan teks deskriptif seperti "Menyatakan Perasaan Sedih dan Kehilangan", "Menyatakan Perasaan Takut dan Kecemasan", "Mengisyaratkan Gejala Fisik", dan 7 label lainnya.

Label-label ini diformat sebagai list Python, lalu dipersiapkan untuk masuk ke tokenizer — persis seperti yang dilakukan pada kalimat pengguna.

WHY — Kenapa penting?

Inilah inti dari filosofi LABAN! Kalau kita merepresentasikan label hanya sebagai angka (0, 1, 2, ...), model tidak "tahu" apa makna dari label itu. Tapi kalau label direpresentasikan sebagai teks yang diproses BERT, model bisa memahami bahwa "Sedih" dan "Kehilangan" secara semantik berhubungan.

Dengan pendekatan ini, model bisa membandingkan "makna kalimat pengguna" dengan "makna teks label" secara langsung di ruang vektor yang sama. Ini juga yang memungkinkan kemampuan *zero-shot* — karena label baru pun bisa diproses langsung tanpa training ulang.

HOW — Bagaimana datanya berubah?

Input: List 10 string teks label intent (Tokenisasi) intent_ids: Shape: (10, max_label_len) intent_mask: Shape: (10, max_label_len)

```
# Di data/data_preparation.py – dilakukan sekali saat inisialisasi
from sklearn.preprocessing import MultiLabelBinarizer

mlb = MultiLabelBinarizer()
mlb.fit(...)
listof_intent = list(mlb.classes_) # 10 label sebagai list string

# Tokenisasi semua label sekaligus
tokenized_intent = tokenizer(
    listof_intent,
    padding=True,
    truncation=True,
    return_tensors='pt'
)

label_ids = tokenized_intent['input_ids'] # Shape: (10, max_label_len)
label_mask = tokenized_intent['attention_mask'] # Shape: (10, max_label_len)
```

3.5.5 EnkodeLabel

WHAT — Apa yang terjadi?

Sekarang 10 teks label tadi dimasukkan ke **BERT encoder kedua** (`self.bertlabelencoder` di `bert_model.py`). Ini adalah encoder yang terpisah dari encoder kalimat pengguna, meskipun keduanya berbagi arsitektur yang sama (IndoBERT). Outputnya kita sebut **clusters** — yaitu matriks berisi 10 vektor, masing-masing merepresentasikan satu label intent.

WHY — Kenapa harus encoder terpisah?

Pertanyaan yang wajar! Kenapa tidak pakai encoder yang sama?

Alasannya: kalimat pengguna dan teks label punya karakteristik yang sangat berbeda. Kalimat pengguna panjang, informal, penuh emosi, kadang slang. Sedangkan teks label pendek, formal, dan deskriptif. Dengan encoder terpisah, masing-masing encoder bisa belajar "spesialisasi" — encoder pertama belajar membaca gaya bicara pengguna, encoder kedua belajar merepresentasikan karakteristik kategori intent. Keduanya berlatih bersamaan sehingga representasinya akhirnya "selaras" di ruang vektor yang sama.

HOW — Bagaimana datanya berubah?

Input: label_ids Shape: (10, max_label_len) label_mask Shape: (10, max_label_len) (BERT Label Encoder) clusters: Shape: (10, 768) <- Matriks 10 baris (satu per label) x 768 kolom

```
# Di bert_model.py – BertEmbedding.forward()
output_label = self.bertlabelencoder(
    input_ids=label_ids,
    attention_mask=label_mask,
    return_dict=True
)

# Ambil representasi [CLS] untuk setiap label
clusters = output_label.pooler_output # Shape: (10, 768)

# clusters[0] = vektor 768-dim untuk label ke-1 ("Menyatakan Perasaan Sedih...")
```

```
# clusters[1] = vektor 768-dim untuk label ke-2 ("Menyatakan Perasaan Takut...")
```

3.5.6 Membangun Vektor Embedding Semantik

WHAT — Apa yang terjadi?

Ini adalah tahap paling krusial secara matematika — di sini kita menghitung **seberapa dekat kalimat pengguna dengan setiap label intent**. Prosesnya menggunakan operasi yang disebut **proyeksi matriks Gram invers**.

Pertama, dihitung **Gram matrix** dari label embeddings: `gram = clusters @ clusters.T` menghasilkan matriks (10, 10) yang menangkap hubungan antar semua pasang label.

Kedua, dihitung **weight matrix**: `weight = pooled_output @ clusters.T` menghasilkan matriks (1, 10) yang menunjukkan seberapa mirip kalimat dengan setiap label (dot product biasa).

Terakhir, weight di-proyeksikan melalui invers Gram: `logits = weight @ inverse(gram)`. Langkah ini yang membuat LABAN spesial — daripada sekadar dot product biasa, proyeksi invers Gram memastikan setiap skor intent dihitung secara **independen terhadap label lain**, bukan relatif terhadapnya.

WHY — Kenapa tidak pakai dot product biasa saja?

Dot product biasa bisa bermasalah kalau dua label semantiknya mirip (misalnya "Sedih" dan "Kehilangan"). Keduanya akan mendapat skor tinggi sekaligus, padahal mungkin pengguna hanya bermaksud satu. Proyeksi invers Gram secara matematis "mengortogonalisasikan" kontribusi setiap label — tiap logit benar-benar merepresentasikan alignment kalimat terhadap label itu secara eksklusif.

HOW — Bagaimana datanya berubah?

`pooled_output`: Shape: (1, 768) `clusters`: Shape: (10, 768) (Operasi Matriks) `gram = clusters @ clusters.T` Shape: (10, 10) `<- Gram matrix` `gram = gram + epsilon*I` Regularisasi Tikhonov `weight = pooled_output @ clusters.T` Shape: (1, 10)

```
# Di bert_model.py - BertEmbedding.forward()
import numpy as np
```

```
# Hitung Gram matrix: seberapa mirip setiap pasang label embedding
gram = torch.mm(clusters, clusters.permute(1, 0)) # Shape: (10, 10)
```

```
# Tambahkan regularisasi Tikhonov supaya gram selalu invertible
gram = gram + torch.eye(gram.size(0), device=gram.device) * 1e-4
```

```
# Hitung weight: dot product kalimat vs. setiap label
weight = torch.mm(pooled_output, clusters.permute(1, 0)) # Shape: (1, 10)
```

3.5.7 VektorEmbedding Semantik

WHAT — Apa yang terjadi?

Sekarang kita menyelesaikan perhitungan logits dan melakukan **scaling**. Logits dihitung dari

proyeksi `weight @ inverse(gram)`, kemudian dikalikan dengan akar kuadrat dari 768 (hidden dimension).

Scaling ini adalah teknik standar dari paper "Attention is All You Need" — ia mencegah nilai dot product menjadi terlalu besar saat dimensi embedding tinggi, yang bisa membuat gradien training tidak stabil.

Hasil akhirnya adalah vektor **logits** berukuran (1, 10) — satu skor mentah untuk setiap dari 10 intent. Nilai ini belum jadi probabilitas (bisa di luar range 0-1), tapi sudah merepresentasikan "seberapa cocok" kalimat pengguna dengan masing-masing intent.

WHY — Kenapa perlu di-scale?

Tanpa scaling, saat dimensi embedding besar (768 dimensi), dot product dua vektor bisa menghasilkan angka sangat besar. Angka besar ini jika masuk ke fungsi Sigmoid akan membuat output mendekati 0 atau 1 secara ekstrim, sehingga gradien menjadi sangat kecil dan training melambat drastis. Scaling dengan $\sqrt{h}$ menstabilkan distribusi nilai.

HOW — Bagaimana datanya berubah?

weight: Shape: (1, 10) gram inverse: Shape: (10, 10) (Perkalian Matriks + Scaling) logits = (weight @ gram_inverse) * $\sqrt{768}$ Shape: (1, 10) <- 10 skor mentah, satu per intent, siap masuk Sigmoid

```
# Di bert_model.py – BertEmbedding.forward() – baris terakhir
logits = torch.mm(weight, torch.inverse(gram)) * np.sqrt(opt.hidden_size)
# opt.hidden_size = 768

# logits.shape: (1, 10)
# Contoh nilai: [-2.3, 0.8, -1.1, 3.2, -0.5, 1.7, -2.0, 2.8, -1.3, 0.4]
# (Nilai bisa negatif, positif, atau nol – belum dinormalisasi)

return logits
```

3.5.8 Probabilitas Multi-Intent

WHAT — Apa yang terjadi?

Logits mentah tadi sekarang dimasukkan ke fungsi **Sigmoid**. Sigmoid mengubah setiap angka — berapapun nilainya — menjadi angka antara 0 dan 1. Ini dilakukan **per-intent secara independen**.

Jadi, kalau logit untuk "Sedih" adalah 3.2, Sigmoid akan mengubahnya jadi ~0.96 (kemungkinan besar intent ini aktif). Kalau logit untuk "Terkejut" adalah -2.3, Sigmoid mengubahnya jadi ~0.09 (kemungkinan kecil intent ini aktif).

Perhatikan: kita **tidak** pakai Softmax. Softmax memaksa semua probabilitas berjumlah 1, cocok untuk klasifikasi *single-label* (hanya satu yang benar). Karena ini *multi-label* (pengguna bisa sekaligus sedih DAN takut), kita butuh Sigmoid yang mengevaluasi setiap intent secara mandiri.

WHY — Kenapa Sigmoid, bukan Softmax?

Ini perbedaan fundamental multi-label vs. multi-class. Di konseling, seseorang bisa merasa sekaligus takut dan sedih — dua intent aktif bersamaan. Sigmoid memungkinkan ini karena setiap output dihitung secara independen, tidak ada "persaingan" antar kelas seperti di Softmax.

HOW — Bagaimana datanya berubah?

logits: Shape: (1, 10) — nilai bisa negatif/positif (Sigmoid per elemen) probs: Shape: (1, 10) — setiap nilai ada di range [0, 1]

Contoh: logits: [-2.3, 0.8, -1.1, 3.2, -0.5, 1.7, -2.0, 2.8, -1.3, 0.4] probs: [0.09, 0.69, 0.25, 0.96, 0.38, 0.85, 0.12, 0.94, 0.21, 0.60]

```
# Di predict.py — Predictor.predict()
logits = model(utterance_ids, utterance_mask, intent_ids, intent_mask)

# Terapkan Sigmoid — BUKAN Softmax!
probabilities = torch.sigmoid(logits).squeeze().cpu().numpy()
# probabilities.shape: (10,) — satu probabilitas per intent

# Saat training, loss function yang dipakai adalah BCEWithLogitsLoss
# (yang sudah include Sigmoid secara internal, lebih numerically stable)
criterion = torch.nn.BCEWithLogitsLoss(reduction='sum')
loss = criterion(logits, labels) # labels adalah multi-hot vector shape (10,)
```

3.5.9 Penentuan Kelas dan Output Multi-Label

WHAT — Apa yang terjadi?

Tahap terakhir ini sederhana tapi krusial: kita menerapkan **threshold** (nilai ambang batas) pada probabilitas yang sudah kita hitung. Threshold di-set ke **0.5** (dikonfigurasi di `config.py` dengan typo kanonik `thresold`).

Setiap intent yang probabilitasnya ≥ 0.5 dianggap aktif (True), sisanya dianggap tidak aktif (False). Hasilnya adalah daftar nama-nama intent yang terdeteksi dari kalimat pengguna.

WHY — Kenapa 0.5?

0.5 adalah titik tengah — artinya model "lebih yakin ada" daripada "tidak ada". Nilai ini bisa disesuaikan: kalau diturunkan (misal 0.3), model jadi lebih sensitif (lebih banyak intent yang ke-trigger, tapi risiko false positive naik). Kalau dinaikkan (misal 0.7), model jadi lebih selektif. Untuk dataset konseling ini, 0.5 ditemukan sebagai keseimbangan yang baik.

HOW — Bagaimana datanya berubah?

probs: [0.09, 0.69, 0.25, 0.96, 0.38, 0.85, 0.12, 0.94, 0.21, 0.60] threshold = 0.5 (Perbandingan dengan threshold) [False, True, False, True, False, True, False, True, False, True] (Ambil nama label yang True) Output: ["Menyatakan Perasaan Percaya", "Menyatakan Perasaan Sedih dan Kehilangan", ...]

```
# Di predict.py — Predictor.predict()
from config import opt

# probabilities sudah dalam bentuk numpy array shape (10,)
detected_intents = []
scores = {}

for i, label_name in enumerate(self.label_names):
    score = float(probabilities[i])
    scores[label_name] = round(score, 4)
    if score >= opt.thresold: # Ingat: typo 'thresold' adalah kanonik,
```

```

jangan diubah!
    detected_intents.append(label_name)

return {
    "pertanyaan": text,
    "inten_terdeteksi": detected_intents,
    "apakah_multilabel": len(detected_intents) > 1,
    "skore": scores
}

# Contoh output:
# {
#   "pertanyaan": "Aku takut dan sedih banget hari ini",
#   "inten_terdeteksi": ["Menyatakan Perasaan Sedih dan Kehilangan",
#                        "Menyatakan Perasaan Takut dan Kecemasan"],
#   "apakah_multilabel": True,
#   "skore": {"Menyatakan Perasaan Sedih dan Kehilangan": 0.94, ...}
# }

```

Catatan Belajar oleh: Mentor AI Framework: PyTorch + HuggingFace Transformers Arsitektur: LABAN (Label-Aware BERT Attention Network) — modifikasi dari Wu et al. (EMNLP 2021) Backbone Aktif: [indobenchmark/indobert-base-p1](#) (IndoBERT, 768-dim) Referensi kode: [models/multilabel/bert_model.py](#), [data/data_preparation.py](#), [models/multilabel/predict.py](#)